

Fundamentos de JavaScript moderno

Variables, tipos de datos, control de flujo, funciones, manejo de errores, DOM y eventos

SEMANA	SESIONES	DURACIÓN	MODALIDAD
01	01 y 02	4 horas	Presencial

CASO PRÁCTICO

Calculadora académica interactiva

Construirás una aplicación web que valida tres notas, calcula el promedio, clasifica el resultado y actualiza la interfaz sin recargar la página.

Curso	Código	Unidad	Periodo
JavaScript Avanzado	100000SI56	Unidad 1: JavaScript	2026 - Ciclo 2 Agosto

Ruta del laboratorio

Esta guía combina explicación conceptual, experimentación breve y construcción guiada. Sigue la secuencia completa; cada bloque prepara el siguiente.

Bloque	Actividad	Evidencia
A	Preparar el entorno y reconocer DevTools	Proyecto y consola operativos
B	Explorar sintaxis, tipos y declaraciones	Resultados en consola
C	Aplicar control de flujo, funciones y errores	Ejercicios verificados
D	Construir la estructura HTML	Interfaz semántica
E	Aplicar estilos CSS	Interfaz responsive
F	Programar la lógica JavaScript	Aplicación funcional
G	Probar, depurar y documentar	Matriz de pruebas
H	Resolver el reto y cerrar	Mejora y reflexión

Ficha del laboratorio

Elemento	Descripción
Logro de la semana	Implementa una interacción web básica aplicando declaraciones, tipos, control de flujo, funciones, manejo de errores, DOM y eventos.
Producto	Aplicación web Calculadora académica organizada en archivos HTML, CSS y JavaScript separados.
Estrategia	Demostración breve, práctica guiada, prueba de casos y reto de extensión.
Trabajo	Individual durante la construcción; contraste de pruebas en parejas al finalizar.
Requisitos	Computadora, Visual Studio Code estable y un navegador moderno con herramientas de desarrollo.

Resultados de aprendizaje observables

- **Explica** la diferencia entre JavaScript, ECMAScript, el navegador y las API web.
- **Declara** datos con `const` y `let`, eligiendo cada palabra clave según la intención de cambio.
- **Decide** mediante condicionales y repite mediante estructuras iterativas sencillas.
- **Encapsula** lógica reutilizable en funciones con parámetros y valor de retorno.

- **Gestiona** errores previstos con throw, try, catch y finally.
- **Interactúa** con el DOM mediante querySelector(), textContent y propiedades seguras.
- **Registra** eventos con addEventListener() y evita manejadores HTML en línea.
- **Verifica** el comportamiento con casos válidos, límites y entradas incorrectas.

Fundamentos teóricos

JavaScript se aprende mejor conectando tres ideas: el lenguaje que expresa la lógica, el navegador que ejecuta el código y el DOM que representa la página que será modificada.

JavaScript y ECMAScript

JavaScript. es un lenguaje de programación de propósito general, dinámico y multiparadigma. En la web se emplea para reaccionar a acciones, validar datos, cambiar el contenido y comunicarse con servicios.

ECMAScript. es la especificación técnica que define la sintaxis y el comportamiento del lenguaje. Esta guía toma como referencia ECMAScript 2026, decimoséptima edición de ECMA-262.

API web. es una capacidad proporcionada por el navegador, como document, console, alert o addEventListener(). Estas API no forman parte del núcleo de ECMAScript, aunque se utilizan desde JavaScript.

Idea clave: JavaScript no es lo mismo que Java. Tampoco es sinónimo de DOM: JavaScript es el lenguaje; el DOM es una API del navegador que el lenguaje puede utilizar.

Cómo se ejecuta el código en el navegador

1. **El navegador lee el HTML.** Con las etiquetas construye una representación en memoria llamada DOM.
2. **El navegador aplica el CSS.** Las reglas visuales se calculan y se combinan con la estructura.
3. **El motor ejecuta JavaScript.** Las sentencias se procesan en orden dentro del contexto correspondiente.
4. **JavaScript consulta o modifica el DOM.** Por ejemplo, obtiene un formulario o cambia el texto de un resultado.
5. **Los eventos activan funciones.** Un submit, un click o un change inicia la lógica registrada para ese evento.

Código 1. Vinculación segura de los archivos externos desde el documento HTML

```
<link rel="stylesheet" href="/css/styles.css">  
<script src="/js/app.js" defer></script>
```

¿Por qué defer?: El navegador descarga el archivo JavaScript sin detener el análisis del HTML y lo ejecuta después de construir el documento. Así, querySelector() puede encontrar los elementos de la página.

Sintaxis mínima y modo estricto

JavaScript distingue mayúsculas de minúsculas. nombre, Nombre y NOMBRE son identificadores diferentes. Las llaves delimitan bloques y la indentación hace visible la estructura. El punto y coma no siempre es obligatorio, pero se utilizará de manera consistente para evitar ambigüedades.

Código 2. Primera instrucción y activación del modo estricto

```
"use strict";  
console.log("Laboratorio iniciado");
```

Modo estricto: La directiva "use strict" ayuda a detectar algunas prácticas propensas a errores. Debe aparecer al inicio del archivo o de una función y no produce una salida visible.

Variables, constantes y tipos de datos

Una variable relaciona un nombre con un valor. En código moderno se parte de const y se cambia a let solo si será necesario reasignar el valor. var existe por compatibilidad histórica, pero su alcance y elevación pueden producir comportamientos menos evidentes para quien empieza.

Código 3. Declaraciones con intención clara

```
const NOTA_APROBATORIA = 12;  
let numeroIntentos = 0;  
  
numeroIntentos = numeroIntentos + 1;  
console.log(numeroIntentos); // 1
```

Tipo	Ejemplo	Qué representa	typeof
string	"Ana"	Texto	"string"
number	17.5	Números enteros y decimales	"number"
bigint	9007199254740993n	Enteros de gran magnitud	"bigint"
boolean	true	Valor lógico verdadero o falso	"boolean"
undefined	let dato;	Ausencia de valor asignado	"undefined"
symbol	Symbol("id")	Identificador único	"symbol"
null	null	Ausencia intencional de valor	"object"*
object	{ nombre: "Ana" }	Colección u objeto complejo	"object"

* typeof null devuelve "object" por una particularidad histórica del lenguaje; null sigue siendo un valor primitivo.

Regla práctica: Usa const para referencias que no serán reasignadas y let cuando el valor deba cambiar. No conviertas todo en let por costumbre.

Conversión y comparación

Los valores de un input se reciben como texto, incluso cuando el atributo type es number. Por eso se realiza una conversión explícita con `Number()`. Después se comprueba el resultado con `Number.isFinite()`.

Código 4. Conversión explícita y comparación estricta

```
const texto = "15";
const nota = Number(texto);

console.log(typeof texto); // string
console.log(typeof nota);  // number
console.log(nota === 15);   // true
console.log(texto === 15);  // false
```

Buena práctica: Prefiere `===` y `!==` para evitar conversiones implícitas durante la comparación. Esta semana solo emplearemos los operadores necesarios; el estudio detallado de operadores corresponde a la semana 2.

Estructuras de control

Condicional. elige qué bloque ejecutar según una expresión booleana.

Bucle. repite un bloque mientras se cumpla una condición o durante una cantidad determinada de iteraciones.

Manejo de excepciones. separa el flujo normal de la respuesta ante una situación inválida o inesperada.

Código 5. Condicional con rutas excluyentes

```
const promedio = 16;

if (promedio >= 18) {
  console.log("Excelente");
} else if (promedio >= 12) {
  console.log("Aprobado");
} else {
  console.log("Requiere refuerzo");
}
```

Código 6. Repeticiones controladas

```
for (let intento = 1; intento <= 3; intento += 1) {
  console.log(`Intento ${intento}`);
}

let pendientes = 2;
while (pendientes > 0) {
  console.log(`Pendientes: ${pendientes}`);
  pendientes -= 1;
}
```

Prevención: Todo bucle while debe modificar la variable relacionada con su condición. Si la condición nunca llega a ser falsa, se produce un bucle infinito.

Funciones y responsabilidad única

Una función agrupa instrucciones que resuelven una tarea. Los parámetros reciben datos y return entrega el resultado. Una función corta, con un nombre verbal y una sola responsabilidad es más fácil de probar y reutilizar.

Código 7. Función con parámetros y retorno

```
function calcularPromedio(nota1, nota2, nota3) {
  return (nota1 + nota2 + nota3) / 3;
}

const resultado = calcularPromedio(18, 16, 17);
console.log(resultado); // 17
```

Parte	Ejemplo	Función
Nombre	calcularPromedio	Expresa la acción que se realiza.
Parámetros	nota1, nota2, nota3	Reciben los datos de entrada.
Cuerpo	{ ... }	Contiene las instrucciones.
Retorno	return promedio	Devuelve un valor a quien llamó la función.
Invocación	calcularPromedio(18, 16, 17)	Ejecuta la función con argumentos concretos.

Errores controlados con try, catch y finally

Código 8. Validación mediante una excepción descriptiva

```
function validarNota(nota) {
  if (nota < 0 || nota > 20) {
    throw new RangeError("La nota debe estar entre 0 y 20.");
  }

  return nota;
}

try {
  console.log(validarNota(25));
} catch (error) {
  console.error(error.message);
} finally {
  console.info("Validación terminada");
}
```

- **throw** interrumpe el flujo y genera una excepción con información útil.
- **try** contiene las instrucciones que podrían producir la excepción.
- **catch** recibe el error y decide cómo comunicarlo o recuperarse.
- **finally** se ejecuta haya o no error; es útil para cierre, limpieza o trazabilidad.

No ocultes errores: Un catch vacío dificulta el diagnóstico. Muestra un mensaje comprensible a la persona usuaria y registra información técnica en la consola durante el desarrollo.

Salidas: consola, alertas y página

Mecanismo	Uso apropiado	Observación
console.log / table / error	Depuración y trazabilidad técnica	La persona usuaria normalmente no ve la consola.
alert()	Aviso bloqueante muy puntual	Interrumpe la interacción; no es adecuado como salida principal.
textContent	Actualizar texto visible en el DOM	Es seguro para texto porque no interpreta HTML.
document.write()	Ejemplos históricos muy básicos	Después de cargar la página puede reemplazar el documento; se evitará.

DOM y eventos

El DOM representa el documento como objetos relacionados. `document.querySelector()` selecciona el primer elemento que coincide con un selector CSS. Después se pueden leer propiedades, cambiar texto o registrar eventos.

Código 9. Selección, evento y actualización del DOM

```
const boton = document.querySelector("#btnSaludar");
const salida = document.querySelector("#salida");

boton.addEventListener("click", () => {
  salida.textContent = "¡Hola desde JavaScript!";
});
```

Evento	Se produce cuando	Uso en formularios
submit	Se intenta enviar un formulario	Procesar el conjunto de datos.
reset	Se restablece el formulario	Limpiar resultados auxiliares.
input	El valor cambia mientras se escribe	Retroalimentación inmediata.
change	Se confirma un cambio de valor	Validación menos frecuente.
click	Se activa un elemento apuntador	Acciones de botones que no envían formularios.

Estándar del laboratorio: El sílabo menciona `onclick` y `onchange`. Los reconocerás, pero implementarás `addEventListener()` porque mantiene el HTML separado de la lógica y permite registrar múltiples manejadores.

Preparación del entorno

El proyecto no requiere Node.js ni dependencias. Solo necesitas un editor y un navegador actual. La estructura se creará manualmente para comprender la responsabilidad de cada archivo.

Crear el proyecto en Visual Studio Code

1. Abre Visual Studio Code.
2. Selecciona Archivo > Abrir carpeta y crea una carpeta llamada calculadora-academica.
3. En el explorador del editor crea las carpetas css y js.
4. Crea index.html en la raíz, styles.css dentro de css y app.js dentro de js.
5. Guarda todos los archivos antes de ejecutar la página.

Estructura esperada del proyecto

```
calculadora-academica/  
├── index.html  
├── css/  
│   └── styles.css  
└── js/  
    └── app.js
```

Nombres exactos: Respeta mayúsculas, minúsculas y rutas. styles.css no es igual que Styles.css en sistemas que distinguen el uso de mayúsculas.

Ejecutar y abrir las herramientas del navegador

1. Abre index.html en el navegador. Para este laboratorio también funciona la apertura directa del archivo.
2. Si tienes una extensión de servidor local, puedes usar Abrir con Live Server para trabajar con recarga automática.
3. Presiona F12 o Ctrl+Shift+I y abre la pestaña Consola.
4. Escribe 2 + 2 y presiona Enter. Debe aparecer 4.
5. Mantén DevTools abierto durante la práctica para observar mensajes y errores.

Si la página aparece vacía: Revisa la pestaña Consola. Un error en rojo suele indicar el archivo y la línea donde se produjo el problema. Corrige el primer error antes de evaluar los siguientes.

Práctica breve en la consola

Ejecuta cada bloque por separado. Predice la salida antes de presionar Enter y contrasta tu explicación con el resultado.

Actividad A: declaraciones y tipos

Código 10. Exploración de valores

```
const curso = "JavaScript Avanzado";
let semana = 1;
const activo = true;
let observacion;

console.log(typeof curso);
console.log(typeof semana);
console.log(typeof activo);
console.log(typeof observacion);

semana = 2;
console.log(semana);
```

- Explica por qué curso y activo se declaran con const.
- Explica por qué semana se declara con let.
- Identifica el valor inicial de observacion y el resultado de typeof.

Actividad B: decisión y repetición

Código 11. Clasificación y contador

```
const nota = 14;

if (nota >= 18) {
  console.log("Excelente");
} else if (nota >= 12) {
  console.log("Aprobado");
} else {
  console.log("Requiere refuerzo");
}

for (let numero = 1; numero <= 3; numero += 1) {
  console.log(`Evidencia ${numero}`);
}
```

- Prueba las notas 18, 12, 11.99 y 20. Anota qué rama se ejecuta.
- Cambia <= 3 por < 3 y explica la diferencia en la cantidad de iteraciones.
- Predice qué ocurriría si numero += 1 desapareciera en un while equivalente.

Actividad C: función y excepción

Código 12. Función validada

```
function duplicarPositivo(valor) {
  if (valor < 0) {
    throw new RangeError("El valor no puede ser negativo.");
  }

  return valor * 2;
}
```

```
}  
  
try {  
  console.log(duplicarPositivo(5));  
  console.log(duplicarPositivo(-1));  
} catch (error) {  
  console.error(error.message);  
}
```

- Identifica el parámetro, los argumentos y el valor de retorno.
- Explica por qué la segunda llamada no produce un número.
- Agrega finally y muestra el texto Prueba terminada.

Construcción guiada de la aplicación

A partir de este punto trabajarás en los tres archivos del proyecto. Copia cada bloque completo, guarda y verifica antes de continuar.

Paso de implementación: estructura HTML

HTML define el significado y el orden del contenido. El formulario ofrece etiquetas asociadas a cada input, botones con tipos explícitos, un área de error anunciable y una sección de resultado actualizable.

Código 13. Contenido completo de index.html

```
<!doctype html>
<html lang="es">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Calculadora académica</title>
    <link rel="stylesheet" href="/css/styles.css">
    <script src="/js/app.js" defer></script>
  </head>
  <body>
    <header class="encabezado">
      <div class="encabezado__contenido">
        <p class="etiqueta">JavaScript · Semana 1</p>
        <h1>Calculadora académica</h1>
        <p>Registra tres notas y obtén un diagnóstico inmediato.</p>
      </div>
    </header>

    <main class="contenedor">
      <section class="tarjeta" aria-labelledby="titulo-formulario">
        <div class="tarjeta__cabecera">
          <div>
            <p class="etiqueta">Registro</p>
            <h2 id="titulo-formulario">Datos del estudiante</h2>
          </div>
          <span class="escala">Escala: 0 a 20</span>
        </div>

        <form id="formNotas" novalidate>
          <div class="campo campo--completo">
            <label for="nombre">Nombre del estudiante</label>
            <input id="nombre" name="nombre" type="text"
              autocomplete="name" placeholder="Ej.: Ana Torres" required>
          </div>

          <div class="rejilla-notas">
            <div class="campo">
              <label for="nota1">Nota 1</label>
              <input id="nota1" name="nota1" type="number"
                min="0" max="20" step="0.01" inputmode="decimal" required>
            </div>

            <div class="campo">
              <label for="nota2">Nota 2</label>
              <input id="nota2" name="nota2" type="number"
                min="0" max="20" step="0.01" inputmode="decimal" required>
            </div>

            <div class="campo">
              <label for="nota3">Nota 3</label>
              <input id="nota3" name="nota3" type="number"
                min="0" max="20" step="0.01" inputmode="decimal" required>
            </div>
          </div>
        </form>
      </section>
    </main>
  </body>
</html>
```

```

        </div>
    </div>

    <p id="mensajeError" class="mensaje-error" role="alert" hidden></p>

    <div class="acciones">
        <button class="boton boton--principal" type="submit">
            Calcular promedio
        </button>
        <button class="boton boton--secundario" type="reset">
            Limpiar
        </button>
    </div>
</form>
</section>

<section id="panelResultado" class="tarjeta resultado"
    aria-labelledby="titulo-resultado" aria-live="polite" hidden>
    <p class="etiqueta">Resultado</p>
    <h2 id="titulo-resultado">Resumen académico</h2>
    <p id="nombreResultado" class="resultado__nombre"></p>

    <div class="resultado__datos">
        <div>
            <span>Promedio</span>
            <output id="promedioResultado" for="nota1 nota2 nota3"></output>
        </div>
        <p id="estadoResultado" class="resultado__estado"></p>
    </div>

    <div class="barra" aria-hidden="true">
        <div id="barraProgreso" class="barra__valor"></div>
    </div>
</section>
</main>

<footer>
    <p>Laboratorio de fundamentos de JavaScript y DOM.</p>
</footer>
</body>
</html>

```

Comprender la estructura HTML

- **doctype y lang** indican HTML moderno y el idioma principal del documento.
- **meta viewport** adapta el ancho a la pantalla de dispositivos móviles.
- **link y script defer** cargan los archivos externos sin mezclar responsabilidades.
- **label for e input id** crean una asociación útil para accesibilidad y selección con JavaScript.
- **type=number, min, max y step** expresan el tipo y los límites esperados, aunque la lógica vuelve a validarlos.
- **novalidate** desactiva los mensajes automáticos del navegador para observar la validación creada en JavaScript.
- **role=alert** permite anunciar errores cuando el mensaje deja de estar oculto.
- **aria-live=polite** informa cambios del resultado sin interrumpir de manera agresiva.
- **output for** representa semánticamente un valor calculado a partir de los tres campos.

Verificación inmediata: Abre la página. Debes ver la estructura sin diseño avanzado. Si aparece un 404 o faltan estilos, revisa las rutas relativas ./css/styles.css y ./js/app.js.

Paso de implementación: presentación CSS

CSS no contiene la lógica del cálculo. Su responsabilidad es producir jerarquía visual, estados reconocibles, foco visible y adaptación a pantallas estrechas.

Código 14. Contenido completo de css/styles.css

```
:root {
  color-scheme: light;
  --color-fondo: #f4f6f8;
  --color-superficie: #ffffff;
  --color-texto: #172033;
  --color-secundario: #5f6b7a;
  --color-primario: #1f4d78;
  --color-primario-oscuro: #153652;
  --color-acento: #d92d20;
  --color-borde: #d7dde5;
  --color-error: #b42318;
  --color-error-fondo: #fef3f2;
  --color-excelente: #147d64;
  --color-logrado: #2563a6;
  --color-aprobado: #8a6100;
  --color-refuerzo: #b42318;
  --sombra: 0 16px 40px rgb(20 36 56 / 10%);
}

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  min-height: 100vh;
  font-family: Inter, "Segoe UI", Arial, sans-serif;
  color: var(--color-texto);
  background: var(--color-fondo);
}

.encabezado {
  padding: 3.5rem 1.25rem 5.5rem;
  color: #ffffff;
  background: linear-gradient(135deg, var(--color-primario-oscuro), var(--color-primario));
}

.encabezado__contenido,
.contenedor,
footer p {
  width: min(100%, 64rem);
  margin-inline: auto;
}

.encabezado h1 {
  max-width: 15ch;
  margin: 0.25rem 0 0.75rem;
  font-size: clamp(2rem, 6vw, 3.5rem);
  line-height: 1;
}

.encabezado p:last-child {
  max-width: 44rem;
  margin: 0;
  color: #dbe8f5;
  font-size: 1.05rem;
}
```

```
}

.etiqueta {
  margin: 0 0 0.35rem;
  color: var(--color-acento);
  font-size: 0.78rem;
  font-weight: 800;
  letter-spacing: 0.12em;
  text-transform: uppercase;
}

.encabezado .etiqueta {
  color: #ffb4ad;
}

.contenedor {
  display: grid;
  grid-template-columns: 1.35fr 0.85fr;
  gap: 1.5rem;
  padding: 0 1.25rem 3rem;
  margin-top: -3rem;
}

.tarjeta {
  padding: 1.75rem;
  border: 1px solid var(--color-borde);
  border-radius: 1rem;
  background: var(--color-superficie);
  box-shadow: var(--sombra);
}

.tarjeta__cabecera {
  display: flex;
  align-items: flex-start;
  justify-content: space-between;
  gap: 1rem;
  margin-bottom: 1.5rem;
}

.tarjeta h2 {
  margin: 0;
  font-size: 1.35rem;
}

.escala {
  padding: 0.4rem 0.65rem;
  border-radius: 999px;
  color: var(--color-primario);
  background: #e9f1f8;
  font-size: 0.78rem;
  font-weight: 700;
  white-space: nowrap;
}

.rejilla-notas {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 1rem;
}

.campo {
  display: grid;
  gap: 0.4rem;
}

.campo--completo {
  margin-bottom: 1rem;
}
```

```
label {
  color: var(--color-secundario);
  font-size: 0.9rem;
  font-weight: 700;
}

input {
  width: 100%;
  padding: 0.8rem 0.9rem;
  border: 1px solid var(--color-borde);
  border-radius: 0.55rem;
  color: var(--color-texto);
  background: #ffffff;
  font: inherit;
}

input:focus {
  border-color: var(--color-primario);
  outline: 3px solid rgb(37 99 166 / 18%);
}

.acciones {
  display: flex;
  flex-wrap: wrap;
  gap: 0.75rem;
  margin-top: 1.35rem;
}

.boton {
  padding: 0.8rem 1rem;
  border: 1px solid transparent;
  border-radius: 0.55rem;
  font: inherit;
  font-weight: 800;
  cursor: pointer;
}

.boton--principal {
  color: #ffffff;
  background: var(--color-primario);
}

.boton--principal:hover {
  background: var(--color-primario-oscuro);
}

.boton--secundario {
  border-color: var(--color-borde);
  color: var(--color-texto);
  background: #ffffff;
}

.mensaje-error {
  margin: 1rem 0 0;
  padding: 0.75rem;
  border-left: 4px solid var(--color-error);
  color: var(--color-error);
  background: var(--color-error-fondo);
  font-weight: 700;
}

.resultado {
  align-self: start;
}

.resultado_nombre {
  margin: 1.25rem 0 0.5rem;
  color: var(--color-secundario);
}
```

```

.resultado__datos {
  display: flex;
  align-items: end;
  justify-content: space-between;
  gap: 1rem;
  margin: 1rem 0;
}

.resultado__datos span {
  display: block;
  color: var(--color-secundario);
  font-size: 0.82rem;
  font-weight: 700;
}

#promedioResultado {
  font-size: 3rem;
  font-weight: 900;
  line-height: 1;
}

.resultado__estado {
  margin: 0;
  font-weight: 900;
}

.estado--excelente { color: var(--color-excelente); }
.estado--logrado { color: var(--color-logrado); }
.estado--aprobado { color: var(--color-aprobado); }
.estado--refuerzo { color: var(--color-refuerzo); }

.barra {
  height: 0.6rem;
  overflow: hidden;
  border-radius: 999px;
  background: #e6eaf0;
}

.barra__valor {
  width: 0;
  height: 100%;
  border-radius: inherit;
  background: linear-gradient(90deg, var(--color-acento), var(--color-primario));
  transition: width 240ms ease;
}

footer {
  padding: 0 1.25rem 2rem;
  color: var(--color-secundario);
  font-size: 0.85rem;
}

@media (max-width: 48rem) {
  .contenedor {
    grid-template-columns: 1fr;
  }

  .rejilla-notas {
    grid-template-columns: 1fr;
  }
}

```

Comprender las decisiones de estilo

- **:root** centraliza colores, sombra y otros tokens reutilizables mediante variables CSS.
- **box-sizing: border-box** hace que padding y borde formen parte del ancho declarado.

- **grid** distribuye formulario y resultado en dos columnas sin posiciones absolutas.
- **:focus** mantiene visible el elemento activo para navegación con teclado.
- **clases de estado** separan el diagnóstico de su color; JavaScript solo agrega la clase correspondiente.
- **transition** anima el cambio de la barra sin alterar el cálculo.
- **media query** reorganiza la aplicación a una columna cuando el ancho es reducido.

Buena práctica: No uses colores como único portador de significado. El estado también se expresa con texto: Excelente, Aprobado o Requiere refuerzo.

Paso de implementación: lógica JavaScript

JavaScript selecciona los elementos una sola vez, distribuye el trabajo en funciones pequeñas y conecta la lógica con dos eventos del formulario.

Código 15. Contenido completo de js/app.js

```
"use strict";

const NOTA_MINIMA = 0;
const NOTA_MAXIMA = 20;
const NOTA_APROBATORIA = 12;

const form = document.querySelector("#formNotas");
const inputNombre = document.querySelector("#nombre");
const inputNota1 = document.querySelector("#nota1");
const inputNota2 = document.querySelector("#nota2");
const inputNota3 = document.querySelector("#nota3");
const mensajeError = document.querySelector("#mensajeError");
const panelResultado = document.querySelector("#panelResultado");
const nombreResultado = document.querySelector("#nombreResultado");
const promedioResultado = document.querySelector("#promedioResultado");
const estadoResultado = document.querySelector("#estadoResultado");
const barraProgreso = document.querySelector("#barraProgreso");

let numeroIntentos = 0;

function convertirNota(input, etiqueta) {
  const texto = input.value.trim();

  if (texto === "") {
    throw new TypeError(`Ingresa ${etiqueta}.`);
  }

  const nota = Number(texto);

  if (!Number.isFinite(nota)) {
    throw new TypeError(`${etiqueta} debe ser un número válido.`);
  }

  if (nota < NOTA_MINIMA || nota > NOTA_MAXIMA) {
    throw new RangeError(
      `${etiqueta} debe estar entre ${NOTA_MINIMA} y ${NOTA_MAXIMA}.`,
    );
  }

  return nota;
}
```

```
function calcularPromedio(nota1, nota2, nota3) {
    return (nota1 + nota2 + nota3) / 3;
}

function obtenerEstado(promedio) {
    if (promedio >= 18) {
        return "Excelente";
    }

    if (promedio >= 15) {
        return "Logro destacado";
    }

    if (promedio >= NOTA_APROBATORIA) {
        return "Aprobado";
    }

    return "Requiere refuerzo";
}

function obtenerClaseEstado(promedio) {
    if (promedio >= 18) return "estado--excelente";
    if (promedio >= 15) return "estado--logrado";
    if (promedio >= NOTA_APROBATORIA) return "estado--aprobado";
    return "estado--refuerzo";
}

function limpiarMensajeError() {
    mensajeError.textContent = "";
    mensajeError.hidden = true;
}

function mostrarResultado(nombre, promedio, estado) {
    nombreResultado.textContent = `Estudiante: ${nombre}`;
    promedioResultado.textContent = promedio.toFixed(2);
    estadoResultado.textContent = estado;
    estadoResultado.className = "resultado__estado";
    estadoResultado.classList.add(obtenerClaseEstado(promedio));

    const porcentaje = (promedio / NOTA_MAXIMA) * 100;
    barraProgreso.style.width = `${porcentaje}%`;
    panelResultado.hidden = false;
}

function mostrarError(error) {
    panelResultado.hidden = true;
    mensajeError.textContent = error.message;
    mensajeError.hidden = false;
    console.error(error);
}

function manejarEnvio(evento) {
    evento.preventDefault();
    limpiarMensajeError();
    numeroIntentos += 1;

    try {
        const nombre = inputNombre.value.trim();

        if (nombre === "") {
            throw new TypeError("Ingresa el nombre del estudiante.");
        }

        const nota1 = convertirNota(inputNota1, "la nota 1");
        const nota2 = convertirNota(inputNota2, "la nota 2");
        const nota3 = convertirNota(inputNota3, "la nota 3");
        const promedio = calcularPromedio(nota1, nota2, nota3);
        const estado = obtenerEstado(promedio);
    }
}
```

```

    mostrarResultado(nombre, promedio, estado);

    console.table({ nombre, nota1, nota2, nota3, promedio, estado });
  } catch (error) {
    mostrarError(error);
  } finally {
    console.info(`Intento de cálculo número ${numeroIntentos}.`);
  }
}

function manejarReinicio() {
  numeroIntentos = 0;
  limpiarMensajeError();
  panelResultado.hidden = true;
  barraProgreso.style.width = "0%";
  inputNombre.focus();
}

form.addEventListener("submit", manejarEnvio);
form.addEventListener("reset", manejarReinicio);

```

Leer el archivo JavaScript por responsabilidades

Bloque	Elementos principales	Responsabilidad
Configuración	NOTA_MINIMA, NOTA_MAXIMA, NOTA_APROBATORIA	Evitar números dispersos y documentar reglas.
Referencias DOM	querySelector()	Conservar acceso a formulario, campos y salidas.
Estado mutable	numeroIntentos	Registrar cuántas veces se intenta calcular.
Validación	convertirNota()	Convertir texto, comprobar número y rango.
Cálculo	calcularPromedio()	Aplicar una fórmula y retornar el resultado.
Decisión	obtenerEstado()	Elegir la etiqueta según el promedio.
Presentación	mostrarResultado(), mostrarError()	Actualizar el DOM sin mezclar el cálculo.
Orquestación	manejarEnvio()	Coordinar lectura, validación, cálculo y salida.
Eventos	addEventListener()	Conectar submit y reset con sus funciones.

Explicación detallada de la lógica

Constantes, referencias y estado

Las reglas académicas se declaran como constantes porque no cambian mientras se usa la página. Las referencias obtenidas con `querySelector()` tampoco serán reasignadas. `numeroIntentos` sí cambia, por eso se declara con `let`.

Comprueba el selector: Si `querySelector()` no encuentra un elemento devuelve `null`. Un id escrito de forma diferente en HTML y JavaScript ocasionará después un error como `Cannot read properties of null`.

Validar y convertir una nota

1. Se obtiene `input.value` y se eliminan espacios con `trim()`.
2. Si el texto queda vacío se lanza `TypeError` con un mensaje específico.
3. `Number(texto)` realiza la conversión explícita.
4. `Number.isFinite(nota)` confirma que el resultado es un número finito.
5. La condición con `||` rechaza valores menores que 0 o mayores que 20.
6. Si no ocurre una excepción, `return` entrega la nota convertida.

Calcular y clasificar

`calcularPromedio()` es una función pura: para las mismas tres entradas produce el mismo resultado y no modifica la página. `obtenerEstado()` usa retornos tempranos; cuando una condición se cumple, la función termina y las siguientes condiciones ya no se evalúan.

Sobre `toFixed(2)`: El promedio se mantiene como `number` durante el cálculo y la clasificación. Solo al mostrarlo se aplica `toFixed(2)`, que produce un texto con dos decimales.

Actualizar el DOM

`mostrarResultado()` escribe con `textContent`, actualiza una clase de estado y calcula el porcentaje visual de la barra. `hidden = false` hace visible la sección. `mostrarError()` realiza el camino alternativo: oculta el resultado, presenta el mensaje y conserva el error técnico en `console.error()`.

Seguridad del texto: Para mostrar datos escritos por una persona se prefiere `textContent`. `innerHTML` interpreta etiquetas y no debe utilizarse con entradas no confiables sin un proceso de sanitización.

Procesar el submit

1. `preventDefault()` evita que el formulario recargue la página.
2. Se limpia cualquier error previo y aumenta el contador de intentos.
3. Dentro de `try` se leen y validan los cuatro campos.

4. Se calcula el promedio y se obtiene el estado.
5. Se actualiza la interfaz y se muestra un resumen técnico con `console.table()`.
6. Si alguna validación lanza una excepción, `catch` llama a `mostrarError()`.
7. `finally` registra el número de intento sin depender del resultado.

Registrar eventos

El formulario escucha `submit`, no solo el click del botón. Esto también cubre el envío con la tecla `Enter` y mantiene la lógica asociada a la intención real: enviar datos. `reset` limpia el estado visual además de los campos.

Código 16. Conexión final entre eventos y funciones

```
form.addEventListener("submit", manejarEnvio);  
form.addEventListener("reset", manejarReinicio);
```

Pruebas y depuración

Una prueba útil indica entradas, acción y salida esperada. Ejecuta la matriz en orden y registra si el comportamiento observado coincide.

Caso	Entradas	Resultado esperado	¿Cumple?
Válido destacado	Ana; 18, 16, 17	17.00 - Logro destacado; panel visible	☐ Sí ☐ No
Límite superior	Luis; 20, 20, 20	20.00 - Excelente	☐ Sí ☐ No
Límite aprobatorio	Rosa; 12, 12, 12	12.00 - Aprobado	☐ Sí ☐ No
Desaprobado	José; 11, 10, 13	11.33 - Requiere refuerzo	☐ Sí ☐ No
Nombre vacío	Sin nombre; 15, 15, 15	Mensaje: Ingresa el nombre...	☐ Sí ☐ No
Nota vacía	Ana; vacío, 15, 15	Mensaje: Ingresa la nota 1	☐ Sí ☐ No
Fuera de rango	Ana; 21, 15, 15	Mensaje: debe estar entre 0 y 20	☐ Sí ☐ No
Reinicio	Cualquier cálculo y Limpiar	Campos y panel quedan limpios	☐ Sí ☐ No

Cómo inspeccionar el programa

1. Abre DevTools y selecciona la pestaña Consola.
2. Realiza un cálculo válido y localiza la tabla generada por `console.table()`.
3. Realiza un cálculo inválido y observa el objeto Error mostrado por `console.error()`.
4. Abre la pestaña Sources o Fuentes y coloca un breakpoint dentro de `manejarEnvio()`.
5. Envía el formulario y avanza línea por línea para observar nombre, nota1 y promedio.
6. Elimina el breakpoint y recarga la página antes de continuar.

Errores frecuentes y corrección

Síntoma	Causa probable	Corrección
No se aplican estilos	Ruta o nombre de <code>styles.css</code> incorrecto	Comprueba <code>./css/styles.css</code> y guarda el archivo.
<code>app.js</code> no produce mensajes	Ruta incorrecta o archivo no guardado	Comprueba <code>./js/app.js</code> , defer y la consola.
Cannot read properties of null	Selector no coincide con un id del HTML	Compara cada <code>#id</code> con exactitud.
La página se recarga	Falta <code>evento.preventDefault()</code>	Añádalo al inicio de <code>manejarEnvio()</code> .
El promedio concatena valores	Se usaron cadenas sin convertir	Aplica <code>Number()</code> antes de sumar.

Síntoma	Causa probable	Corrección
No aparece el resultado	panelResultado continúa oculto	Verifica hidden = false en mostrarResultado().
El estado conserva un color anterior	No se restableció className	Asigna la clase base antes de agregar la nueva.
SyntaxError	Llave, paréntesis o comilla sin cerrar	Revisa desde la línea indicada y formatea el archivo.

Reto de extensión

Elige un reto. No cambies la estructura general ni elimines las validaciones existentes. Prueba nuevamente después de cada modificación.

Reto 1: nueva regla de diagnóstico

Añade el estado En riesgo para promedios menores que 8 y conserva Requiere refuerzo entre 8 y menos de 12. Crea una clase CSS específica para el nuevo estado.

- Pista: ordena las condiciones desde el valor más alto hacia el más bajo.
- Agrega casos para 8.00 y 7.99.

Reto 2: validación al cambiar de campo

Registra el evento change en cada campo de nota para advertir de inmediato cuando el valor esté fuera del rango. Reutiliza convertirNota() y no dupliques la regla de 0 a 20.

- Pista: el objeto evento permite acceder al elemento con evento.currentTarget.
- El mensaje debe desaparecer cuando el valor vuelva a ser válido.

Reto 3: cuarta nota

Agrega una cuarta nota en HTML y JavaScript. Ajusta calcularPromedio(), la salida semántica for del elemento output y la rejilla CSS.

- Pista: actualiza fórmula, selector y validación; no basta con agregar el input.
- Incluye una prueba donde las cuatro notas sean 20.

Preguntas de comprobación

1. ¿Qué diferencia existe entre JavaScript y ECMAScript?
2. ¿Por qué `input.value` requiere conversión antes de una suma?
3. ¿Cuándo corresponde `const` y cuándo `let`?
4. ¿Qué ventaja ofrece `===` frente a `==` en esta etapa de aprendizaje?
5. ¿Qué responsabilidad tiene `return` dentro de una función?
6. ¿Qué diferencia práctica existe entre `try` y `catch`?
7. ¿Por qué `textContent` es preferible para mostrar el nombre ingresado?
8. ¿Por qué se escucha `submit` y no únicamente `click`?
9. ¿Qué problema resuelve `defer` en la etiqueta `script`?
10. ¿Qué pruebas demuestran que los límites 0, 12 y 20 funcionan correctamente?

Respuestas orientadoras

- ECMAScript especifica el lenguaje; JavaScript es una implementación y ecosistema utilizado en navegadores y otros entornos.
- `input.value` es string; `Number()` evita concatenación y hace explícita la intención numérica.
- `const` expresa que la referencia no será reasignada; `let` se reserva para datos que cambian.
- La comparación estricta evita conversión implícita entre tipos diferentes.
- `return` entrega un resultado y termina la ejecución de la función.
- `try` delimita operaciones susceptibles de fallar; `catch` procesa la excepción producida.
- `textContent` trata la entrada como texto y no como marcado HTML.
- `submit` representa el envío del formulario y también se activa con Enter.
- `defer` pospone la ejecución hasta que el HTML esté analizado, sin bloquear su lectura.
- Deben ejecutarse casos de frontera y compararse con una salida esperada explícita.

Cierre y reflexión

Síntesis: La aplicación funciona porque cada capa cumple una responsabilidad: HTML estructura, CSS presenta y JavaScript valida, decide, calcula y reacciona a eventos.

- ¿Qué error encontraste y qué evidencia te permitió localizarlo?
- ¿Qué función sería más fácil de probar de forma aislada y por qué?
- ¿Qué mejorarías para una aplicación académica real con muchos estudiantes?

Glosario esencial

Término	Definición
API web	Capacidad expuesta por el navegador y utilizable desde JavaScript.
Argumento	Valor concreto entregado al invocar una función.
DOM	Representación en objetos del documento HTML.
Evento	Notificación de que ocurrió una interacción o cambio.
Excepción	Situación que interrumpe el flujo normal y puede ser capturada.
Función	Bloque reutilizable que puede recibir parámetros y retornar un valor.
Parámetro	Nombre declarado por una función para recibir un dato.
Selector CSS	Expresión utilizada para identificar elementos, también desde <code>querySelector()</code> .

Término	Definición
Tipo dinámico	Característica por la cual el tipo pertenece al valor y se determina durante la ejecución.
Validación	Comprobación de que un dato cumple las reglas antes de procesarlo.